National University of Sciences and Technology
School of Electrical Engineering and Computer Science
Department of Computing

CS 878 Deep Learning

Fall 2023

Assignment 1

Multipurpose MLP model for Classification and Regression

Announcement Date: 5th Oct 2023

Due Date: 15th Oct 2023 at 11:59 pm (on LMS)

Instructor: Dr. Muhammad Moazam Fraz

Assignment 1

The objective of this assignment is to give the students a hands-on experience in implementing and training artificial neural networks. The students will be demonstrating the versatility of such networks by implementing a single model that can perform both classification as well as regression tasks on a predefined dataset. The students are expected to design the network, train it for the three tasks, and then finally assess its performance.

Provided Material:

A Starter Jupyter notebook has been provided on LMS. The dataset link is in the notebook.

The link of notebook is here.
https://colab.research.google.com/drive/1X-PtRdjBAC__QkWsuTVmrov5mHpnOoct
Its Read only copy. You need to create your own copy to work in this.

The students are needed to work on the Jupyter Notebook on Colab and prepare a report on their working as mentioned in part 4 of this assignment.

Deliverables:

Create a ZIP file of Jupyter Notebook and PDF Report
Zip file should be named as <YOUR CMS ID>_<Your Name>_A1.zip
Submit the ZIP File on LMS

Dataset:

The given dataset is a large-scale face dataset with long age span (range from 0 to 116 years old). The dataset consists of over 20,000 face images with annotations of age, gender, and ethnicity. We will be using only 7000 of the 20,000 images. The dataset in this assignment is going to be used for the following task:

Given the pixels of an image, perform age estimation, ethnicity classification and gender classification for the image using a single neural network.

Number of Input features in an image: 2304 (48x48 pixel values)
Number of Labels for each input image: 3
Labels:

1. Age (Regression)
2. Ethnicity (Multiclass classification)
3. Gender (Binary Classification)

Instructions:

The dataset for this assignment is already cleaned and provided in a CSV format for you with images included as pixel values.
The 4 columns contain the following information:

1. **[age]** is an integer from 0 to 116, indicating the age. You can convert it to float for regression predictions
2. **[gender]** is either 0 (male) or 1 (female)
3. **[race]** is an integer from 0 to 4, denoting White, Black, Asian, Indian, and Others (like Hispanic, Latino, Middle Eastern).
4. **[pixels]** The 4th column contains the pixels of the images flattened to a single dimension. Each of the original image is of size 48x48 and the flattened image vector consists of 2304 pixel values.

An initial processing has already been done using pandas in the starter code Jupyter notebook. Hence you have a data frame ready for your assignment. Please use the "df_sampled" dataframe in the notebook.

Part 1: Dataset Creation and Data Splitting (10 points):

Split the dataset into training (70%), validation (15%), and testing (15%) subsets. Try to standardize your input features for faster training times and possible better performance.
Cleary show how you manage datasets and dataloaders with clear comments.
Note: The code should be modular. Documentation, and comments in the code have their own marks.

Part 2: Building a Unified Neural Network (40 points)

(i): Neural Network Architecture (30 points)

Design a single feedforward neural network that can handle all three tasks of age prediction, ethnicity classification and gender classification.
The diagram in *Figure 1* shows an example of the type of neural network that you are going to implement. There are two parts of the network:

a. **Feature Extractor:** Firstly, there is the main network denoted by the symbol **(a)** which performs hierarchical features extraction from your input image. When an image is input to this network, it extracts useful features from the image starting from low-level features in the initial layers to high-level abstract features in its later hidden layers. Hence, based on its use let's call this network the "**Feature Extractor**". The feature extractor gives us a **feature** or **image representation** vector from its final hidden layer that represents high level features present in that image.

b. **Output Heads:** We can use the representation vector from the feature extractor to perform multiple tasks by providing it to the respective heads of our 3 three tasks at hand, as shown in the image. Each head can be a **single output layer,** or it can be a **subnetwork followed by the respective output layer**. The subnetwork is a small neural network with a few hidden layers. The output layers in each head used would

be specific to its designated task. For example, in case of ethnicity classification head you would have 5 nodes in its output layer.

Some other points to note:

- You do not need to follow the exact pattern of choosing hidden layers as provided in this diagram.
- You can experiment with different combinations by choosing different sizes of hidden layers in your feature extractor network as well as your output heads.
- Remember to use activation functions like ReLU in your hidden layers.
- You may also try using batch normalization layers and check if they improve the performance of your network.

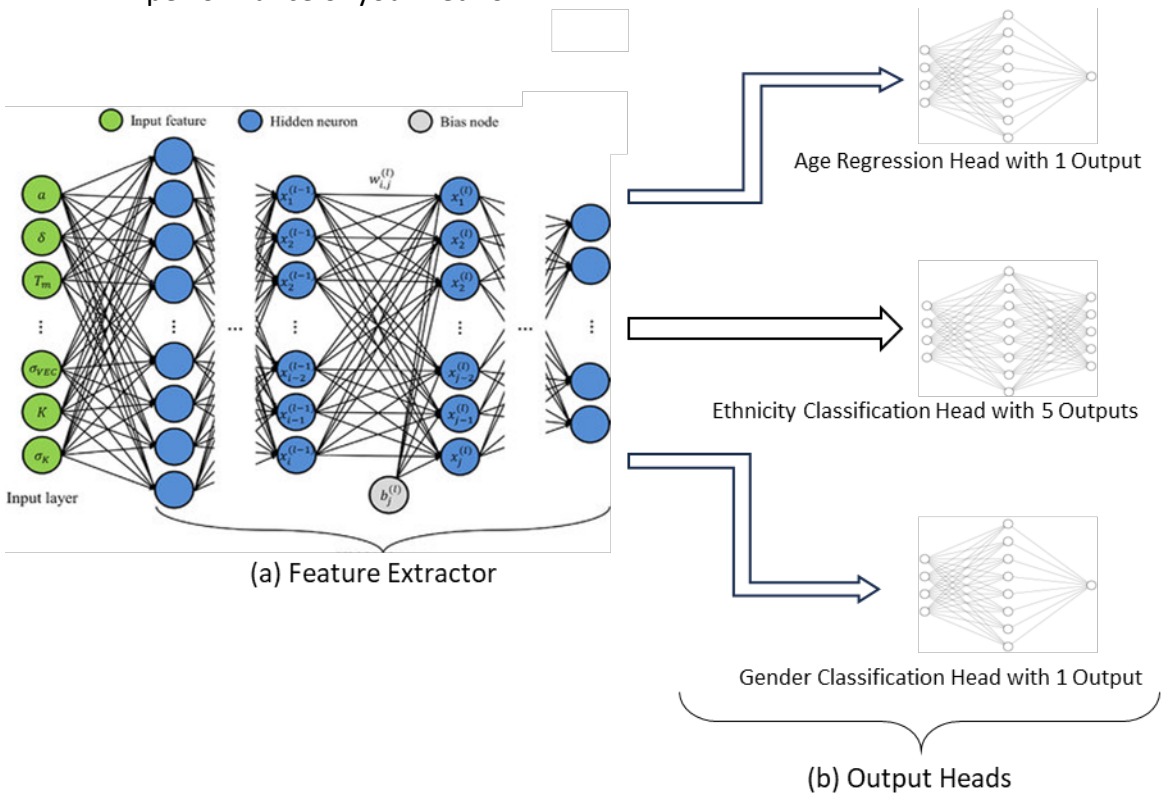


Figure 1

(ii): Model Training (20 Points)

Train the unified neural network on the training subset of the data created in Part 1 while validating its performance on the validation set after each epoch.

For the two classification and single regression tasks, use appropriate losses. Use an optimizer of your choice.

When you have a single neural network that is designed to perform multiple tasks simultaneously, it's essential to carefully consider how to combine the losses for each task. This process is known as multi-task learning. The choice of how to combine the losses depends on the relative importance of each task and the nature of the losses (e.g., regression vs. classification). Here are a few common approaches:

1. **Simple Summation of Losses:** You can simply add the losses for each task without any weighting. For example, if you have a mean squared error (MSE) loss

for age prediction and cross-entropy losses for ethnicity and gender classification, you would sum these losses as follows:

Total Loss = Age_Loss + Ethnicity_Loss + Gender_Loss

This approach treats all losses equally, which might not be ideal in your case if the age loss dominates the other losses due to its nature as a regression task.

2. **Weighted Losses:** Assign different weights to each loss based on their relative importance. In your case, you can give a lower weight to the age loss to account for its larger scale. For example:

Total Loss = w1 * Age Loss + w2 * Ethnicity Loss + w3 * Gender Loss

You can experiment with different weight values to find the right balance that works for your specific problem.

The choice between these methods depends on your understanding of the tasks and their relative importance.

Note: It is not absolute that one of the above method of unifying the losses would perform better than the other. So you are free to use either of them and if you gain good performance, you will receive the due marks. It is encouraged that you try both methods and check which one performs better.

(iii): Performance Monitoring (10 Points)

Keep track of the training and validation losses and accuracies during the training for each task. Visualize the training and validation curves (e.g., loss vs. epochs, accuracy vs epochs) for the three tasks.

Note: The code should be modular. Documentation, and comments in the code have their own marks.

Part 3: Evaluation (10 points)

(i): Classification Evaluation

Evaluate the model's classification performance on the testing data. Calculate **accuracy** and **losses** for the two classification tasks on the test sets. Finally, visualize the confusion matrix.

(ii): Regression Evaluation

Evaluate the model's regression performance for age prediction on the testing data by calculating the **MSE** (Mean Squared Error).

Note: The code should be modular. Documentation, and comments in the code have their own marks. Model performance in this case will be noted. Drastically low performances values on the test set would result in deduction of marks.

Part 4: Conclusion and Discussion (20 points)

Task 8: Report

Write a report summarizing the assignment, including data generation, network architecture, training process, and evaluation results. Other things to include are as follows:

- Discuss the advantages and challenges of using a single network for both classification and regression tasks as well as potential real-world applications of such networks.
- Which settings for your model creation and the training process performed the best for you? Like how many hidden layers in the feature extractor and the subnetwork gave you good performance on the test set? What learning rate did you choose for your training. In short, what type of settings worked better for you and why?

Submission:

Students should submit their code (**Jupyter Notebooks**) and the report as a **PDF** document in a single zipped file on LMS. Once you have submitted your zipped file, be sure to check whether the file has been uploaded correctly or not by redownloading your submission and opening the jupyter notebook as well as the PDF report.

Grading:

The grading rubrics are mentioned in each section, however, you should note that the rubrics in general involve code correctness, modularity, documentation and clear comments.

This assignment will challenge you to create a versatile network capable of handling both classification and regression tasks, encouraging you to think critically about the advantages and limitations of these approaches in different scenarios.

End